6 W.A.C++ program to Implement Linear Search

```
#include <iostream>
Using namespace std;
int main(){
    int n, Key;
    cout<<" Enter the number of elements : ";
    cin>>n;
    int arr[n];
    cout<<" Enter the array element : ";
    for(int i=0; i<n; i++){
      cin>> arr[i];
}

    cout<<" Enter the Key to Search : ";
    cin>> Key;
    int position = -1;
    for(int i=0; i<n; i++){
      if( arr[i] == Key){
        Position = i+1;
        break;
}
}
  if (position != -1){
    cout<<" Key found at position : "<<
      Position <<endl;
} else {
    cout<<" Key not found "<<endl;
}}
```

Write a

Sample O/P :-

Enter the number of elements : 5
Enter the array elements : 10 20 30 40 50
Enter the Key to Search : 30
Key found at position : 3

Write a C++ program for Binary Search :-

```
#include <iostream>
using namespace std;
int main(){
 int arr[] = {10,20,30,40,50};
int key, low = 0, high = 4, mid;
cout << " Enter key to Search : ";
cin >> key;
while( low <= high){
      mid = (low + high)/2;
   if ( arr[mid] == key){
     cout<< " key found at position " << mid+1;
      return 0;

} else if (arr[mid] < key)
      low = mid + 1;
   else   high = mid-1;

}  cout << " key not found ";
     return 0;

}
```

Sample Output :-

Enter Key to Search : 30
Key found at position 3

```
high = n-1, mid, binary = -1;
cout<<" Enter Key to Search: ";
cin>> Key;
for( int i=0; i<n; i++)
    if ( arr[i] == Key)
        binary = i+1;
        break;
}
while ( low <= high)
    mid = (low + high) / 2;
    if ( arr[mid] == Key)
        binary = mid + 1;
        break;
    } else if ( arr[mid] < Key)
        low = mid + 1;
    else
        high = mid - 1;
```

```
if (linear != -1)
    cout<<" Linear Search : Found at position "<<
    <<endl;

else
    cout<<" Linear Search : Key not found "<<endl;

if ( binary != -1)
    cout<<" Binary Search : Found at position "<<
    binary << endl;

else
    cout<<" Binary Search: Key not found "<<endl;
    return 0;
}
```

Sample O/p :-

- Enter Key to Search : 30
- Linear Search : Found at position 3
- Binary Search : Found at position 3

Program : Bubble Sort in Ascending Order

```
#include<iostream>
Using namespace Std;
int main(){
  int arr[] = {5,3,8,2,1};
  int n=5, temp;
for(int i=0; i<n-1; i++){
  for(int j=0; j<n-i-1; j++){
      if(arr[j]> arr[j+1]){
          temp = arr[j];
          arr[j] = arr[j+1];
          arr[j+1] = temp;
  }
    cout<<"Pass "<<i+1<<" : ";
    for(int k=0; k<n; k++){
        cout<<arr[k]<<" ";
    cout<<endl;
}
    } return 0;
}
```

Sample O/P :-

Pass 1 : 3 5 2 1 8

Pass 2 : 3 2 1 5 8

Pass 3 : 2 1 3 5 8

Pass 4 : 1 2 3 5 8

Write a C++ program : Bubble Sort in descending Order

```
#include <iostream>
Using namespace std;
int main(){
int arr[] = {5,3,8,2,1};
int n=5, temp;

for( int i=0 ; i<n-1 ; i++){
    for( int j=0; j<n-i-1 ; j++){
        if ( arr[j] < arr[j+1] ){
            temp = arr[j];
            arr[j] = arr[j+1] ;
            arr[j+1] = temp ;

}}}

cout<<" Array in descending Order : ";
for( int i=0; i<n ; i++)
    cout<<" arr[i]<<" ";

}
```

Sample O/P :-

Array in descending Order : 85321

W.A.C++ program : Insertion Sort in Ascending Order

```
#include<iostream>
Using namespace std;
int main(){
  int arr[] = {5,3,8,2,1};
  int n=5, key, j;
for( int i=1; i<n; i++){
      key = arr[i];
      j= i-1;
while( j>=0 && arr[j]>key){
      arr[j+1] = arr[j];
       j--;
}
 arr[j+1]= key;
 cout<<"Pass "<<i<<" : ";
for( int k=0; k<n; k++)
      cout<< arr[k] <<" ";
      cout<< endl;
}
 return 0;
}
```

Simple O/P :-

Pass 1 : 35 82 1

Pass 2 : 3 5 8 2 1

Pass 3 : 2 35 8 1

Pass 4 : 1 2 3 5 8

WAP : Merge Sort

```
#include <iostream>
Using namespace std;
Void merge (int arr[], int left, int mid, int right){
    int temp[100], i=left, j=mid+1, k=0;
while (i<=mid && j<=right){
    if (arr[i]<=arr[j])
        temp[k++] = arr[i++];
    else
        temp[k++] = arr[j++];
}while(i<=mid)
    temp[k++]=arr[i++];
while(j<=right)
    temp[k++]=arr[j++];
for(i=left, k=0; i<=right; i++,k++)
    arr[i]=temp[k];
}
Void mergesort(int arr[], int left, int ri
{   if(left<right){
```

```
        mergeSort(arr, left, mid);
        mergeSort(arr, mid+1, right);
        merge(arr, left, mid, right);
    }
};

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;

    mergeSort(arr, 0, n-1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Sample O/P 1-

Sorted array : 3 9 10 27 28 38 43 82

W. A. P : Quick Sort

```
#include <iostream>
Using namespace std;
int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;
    for(int j = low; j < high; j++){
        if(arr[j] < pivot){
            i++;
            Swap(arr[i], arr[j]);
        }
    }
    Swap(arr[i+1], arr[high]);
    return i+1;
}
Void quickSort(int arr[], int low, int high){
    if(low < high){
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi, -1);
        quickSort(arr, Pi+1, high);
    }
}
```

```
int main(){
int arr[] = { 10, 7, 8, 9, 1, 5 };
  int n=6;
quickSort( arr, 0, n-1);
Cout<<" Sorted array: ";
for( int i=0; i<n; i++)
      Cout<< arr[i] <<" ";
  return 0;
}
```

Sample O/P :-

Sorted array : 1 5 7 8 9 10

W.A.P Quick Sort After Each partition

```
#include<iostream>
Using namespace std;
int partition (int arr[], int low, int high){
    int pivot = arr[high];
  int i = low-i;

for ( int j= low; j<high; j++){
    if ( arr[j]< pivot){
      i++;
    Swap( arr[i], arr[j]);

};
  Swap( arr[i+1], arr[high]);
Cout<<"After partition:";
for ( int K=0; K<6; K++)
    Cout<< arr[K]<<" ";
    Cout<< endl;
    return i+1;
}
```

```
if (low < high){
int pi = partition(arr, low, high){
  if ( low<high){
    int pi = Partition(arr, low, high);
    quickSort(arr, low, pi -1);
    quickSort(arr, pi+1, high);
}}

int main(){
  int arr[]={10,7,8,9,1,5};
  int n=6;
quickSort(arr, 0,n-1);
Cout<< "Sorted Array :";
for( int i=0; i<n; i++){
    Cout<< arr[i]<<" ";
}
```

W.A.P Menu - Driven Linear Search & Binary Search

```
#include <iostream>
Using namespace std;
int main(){
int arr[] = {10, 20, 30, 40, 50};
int n=5, key, choise, pos=-1;
Cout<<"1, Linear Search"<<endl;
Cout<<"2, Binary Search \n";
Cout<<"Enter your choice:";
Cin>> Choise;
Cout<<"Enter Key to Search:";
Cin>> key;
if(choice==1){
for(int i=0; i<n; i++){
if(arr[i]== key){
Pos(i+1);
break;
}}}
else if(choice==2){
int low=0, high=n-1;
```

```
if ( arr[mid] == Key) {
    Pos = mid + 1;
    break;
} else if (arr[mid] < Key)
    low = mid + 1;
else
    high = mid - 1;
}
} else {
    cout << "Invalid choice!";
    return 0;
}
if (pos != -1)
    cout << "Key found at position " << pos;
else
    cout << "Key not found";
}
```

W.A.P : Merge Sort and Binary Search

```
#include <iostream>
Using namespace Std;
Void merge( int a[], int l, int m, int h){
  int temp[100]; i=1, j=m+1, K=0;
While( i<=m && j<=h){
    if (a[i]<= a[j])
      temp [K++] = a[i++];
    else
      temp[K++]= a[j++];
}
  While (i<=m) temp [K++] = a[i++
  While (j<=h) temp [K++] = a[j++];
  for( j=1; K=0; i<=h; j++, K++)
    a[i] = temp[K];
}
Void merge Sort (int a[], int l, int h){
  if(l<h){
    int m = (l+h) /2;
    merge Sort(a, l, m);
    merge Sort(a, m+l, h);
    merge(a, l, m, h);
```

```
    if (a[m] == Key) return m+1;
    if (a[m] < Key) l = m+1;
    else h = m-l;
  }
    return -1;
}
int main() {
int a[] = {40, 10, 50, 20, 30};
int n=5, Key;
mergeSort(a, 0, n-1);
cout << " Sorted array : ";
for( int i=0; i<n; i++)
    cout << a[i] << " ";
cout << " In Enter element to Search : ";
cin >> Key;
```

```
int pos = binarySearch(a, n, key);
if (pos != -1)
    cout<<"Element found at position "<<pos;
else
    cout<<"Element not found";
return 0;
}
```

Sample O/P :-

Sorted array : 10 20 30 40 50

Enter element to search : 30

Element found at position 3

```
using namespace std;
void bubbleSort(int a[], int n, int &c){
  for(int i=0;i<n;i++){
    for(int j=0;j<n-i-1;j++){
      c++;
    }
    if(a[j]>a[j+1])
      swap(a[j],a[j+1]);
  }
}

void SelectionSort(int a[], int n, int &c){
  for(int i=0;i<n-1;i++){
    int min=i;
    for(int j=i+1;j<n;j++){
      c++;
      if(a[j]<a[min])
        min=j;
    }
    swap(a[i],a[min]);
  }
}

int main(){
```

```
int a[] = {40, 10, 30, 20, 50};
int b[] = {40, 10, 80, 20, 50};
int n = 5, c1 = 0, c2 = 0;

bubbleSort(a, n, c1);
SelectionSort(b, n, c2);
cout<<"Bubble Sort Comparision : "<<c1<<endl;
cout<<"Selection Sort Comparision : "<<c2<<endl;

return 0;
}
```

Simple O/P :-

Bubble Sort Comparision : 10
Selection Sort Comparision : 10

W.A.P Three Sorting Algorithms & their time Complexities.

```
#include <iostream>
Using namespace std;
Void bubbleSort( int a[], int n){
for( int i=0; i<n-1; i++)
for( int j=0; j<n-i-1; j++)
if( a[j]> a[j+1])
Swap( a[j], a[j+1]);
}
Void InsertionSort( int a[], int n){
for( int i=1; i<n; i++){
int Key = a[i], j=i-1;
While( j>=0 && a[j] >Key){
a[j+1] = a[j];
j--;
}
a[j+1] = Key;
}
}
Void SelectionSort(int a[], int n){
for( int i=0; i<n-1; i++){
int min =i;
```

```
for( int j= i+1 ; j<n ; j++)
    if( a[j]<a[min] )
        min = j;
    Swap(a[i], a[min]) ;
}

int main(){
    cout<<" Bubble Sort : Best (O(n)), Average
        O(n^2), Worst O(n^2) ";

    cout<<" Insertion Sort : Best O(n), Average
        O(n^2), Worst O(n^2) "<<endl;

    cout<<" Selection Sort : Best O(n^2),
        Average O(n^2), Worst O(n^2)
        "<<endl;

    return 0;
}
```

Simple O/P :-

Bubble Sort : Best O(n), Average O(n^2), Worst O(n^2)
Insertion Sort : Best O(n), Average O(n^2), Worst
O(n^2)
Selection Sort : Best O(n^2), Average (O(n^2),
Worst O(n^2)